The way to declare lists in go: (This declaration called *slice*)

```
var taskList = []string{
        "1. Watch the crash course of Go",
        "2. Build network automation tools with Go",
        "3. Build Microservices Applications With gRPC",
        "4. Build my Applications using Go",
    }
```

****

****

If we declare variables without using them, then the result is:

```
C:\Tests\Go\From-Youtube-01>go run main.go
# command-line-arguments
.\main.go:16:2: declared and not used: task05

C:\Tests\Go\From-Youtube-01>
```

****

****

If we define the size of slice, then it's Array:

****

We declare arrays: *var list = [size]variable_type {} // it is constant size*

- **Ex**: *var tasks = [20]string {"...", "..."}*

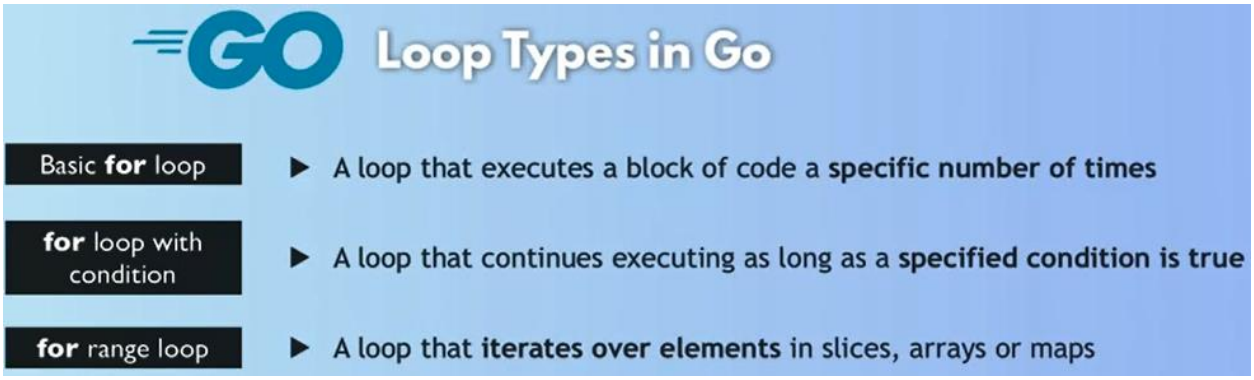
****

****

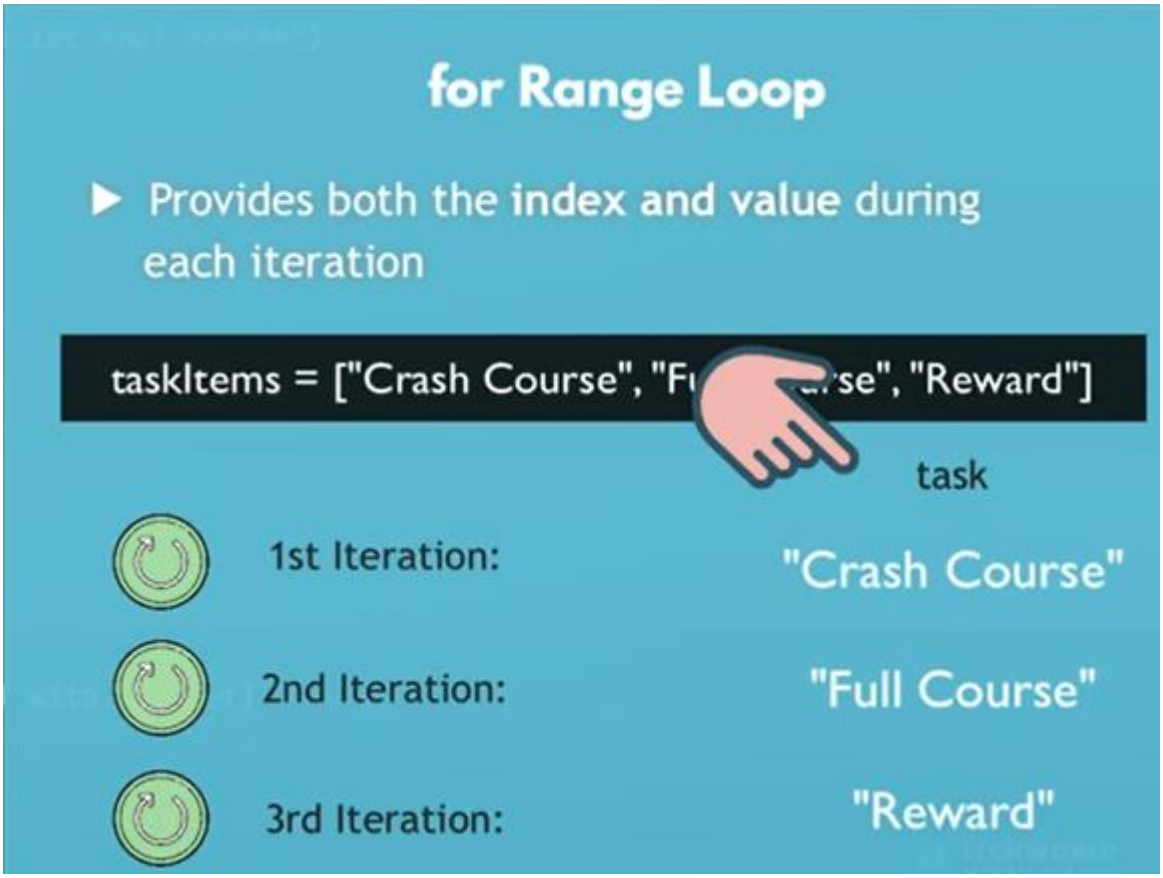
To print multiple variables, using fmt:

```
fmt.Println("*** Task Of Items ***")
fmt.Println("Tasks", taskSlice)
```

****



****



****

Note: *for* and *range* is reserved words in Golang:

```
for index, task := range taskSlice {
        fmt.Println("The index is:", index, ", and task is: ", task)
}
```

****

Using _ to *ignore variables*, especially for index of *for && range loops*:

Here we use temp variables with for-loop (task-variable)

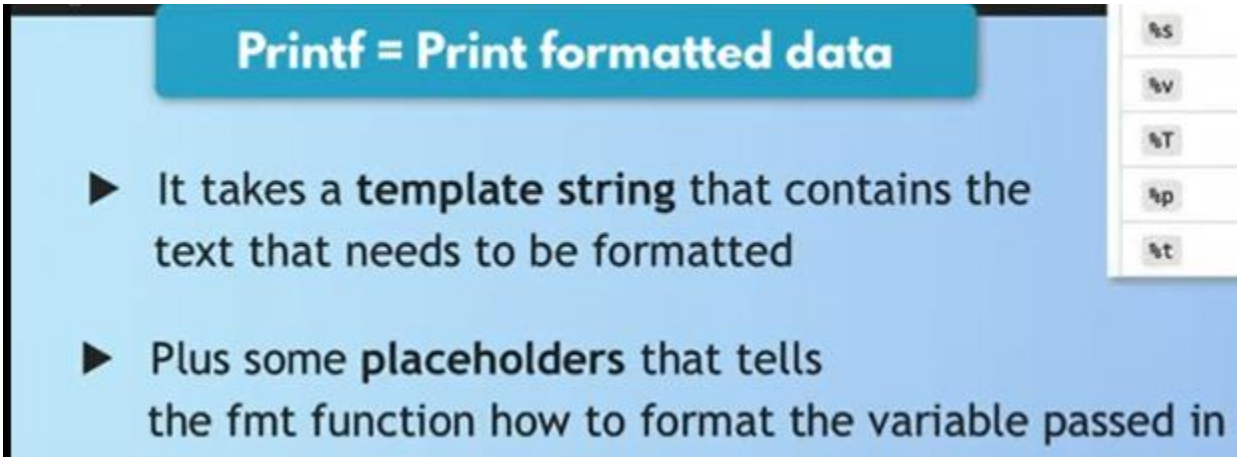
****

Common Placeholders

Here are so mon format specifiers used with `fmt.Printf`:

Placeho	Description	Example Usage
`%d`	Decimal integer	`fmt.Printf("%d", 42)`
`%f`	Floating-point number	`fmt.Printf("%f", 3.1`
`%s`	String	`fmt.Printf("%s", "He`
`%v`	Default format for any value	`fmt.Printf("%v", myV`
`%T`	Type of the value	`fmt.Printf("%T", myV`
`%p`	Pointer address	`fmt.Printf("%p", &my`
`%t`	Boolean value	`fmt.Printf("%t", tru`

****

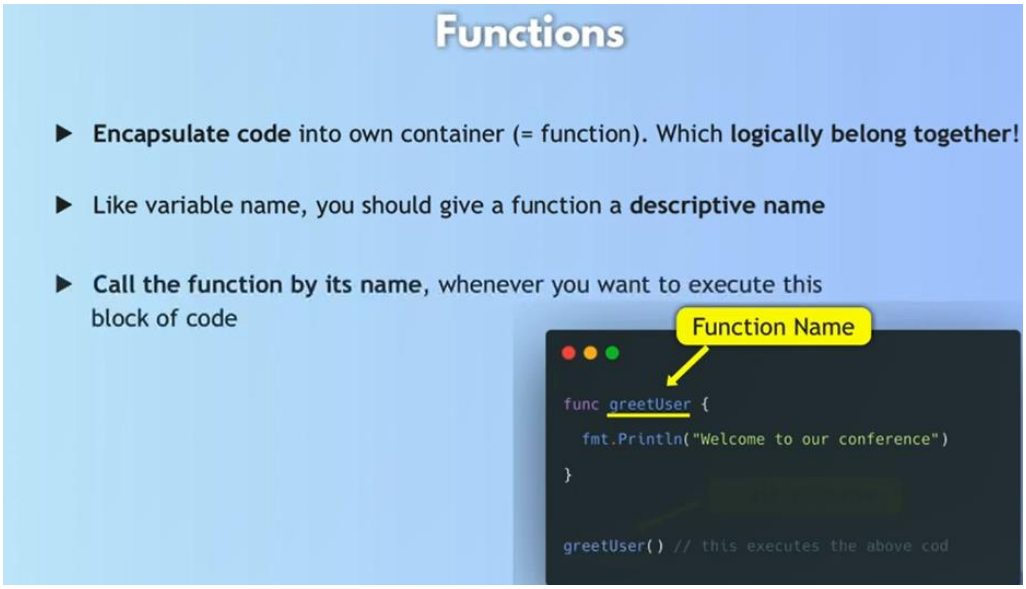


****

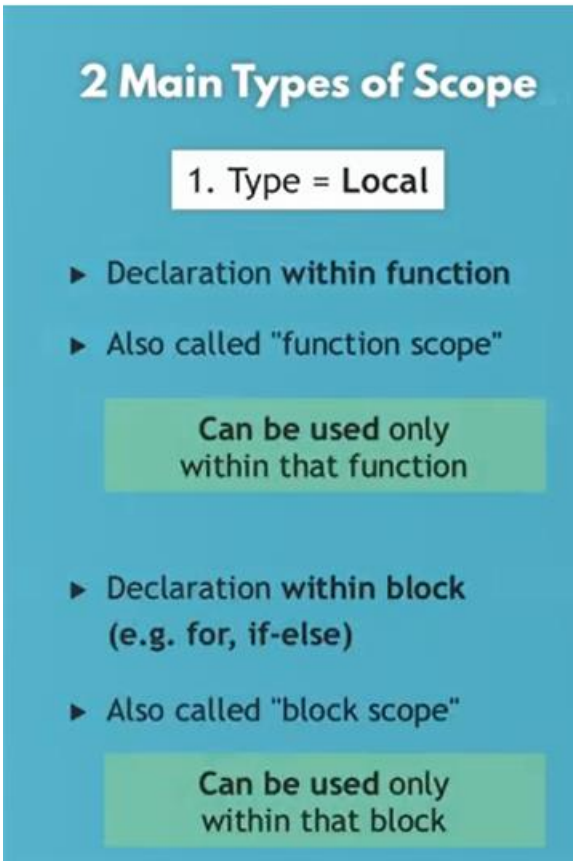
We Can use format string using fmt:

```
for index, task := range taskSlice {
   fmt.Printf("%d. %s\n", index+1, task)
}
```

****


Functions
Encapsulate code into own container (= function). Which logically belong together!
Like variable name, you should give a function a descriptive name
Call the function by its name, whenever you want to execute this block of code
Function Name
func greetUser {
fmt.Println("Welcome to our conference")
}
greetUser() // this executes the above cod

****


2 Main Types of Scope
1. Type = Local
Declaration within function
Also called "function scope"
Can be used only within that function
Declaration within block (e.g. for, if-else)
Also called "block scope"
Can be used only within that block

****

****

To define function with params:

```
func printTasks(taskItems []string) {
    fmt.Println("##### Welcome to our Todolist App! #####")

    for index, task := range taskItems {
        fmt.Printf("%d. %s\n", index+1, task)
    }
}
```

****

To make the function return a value, we must change the signature of the function:

Note: If the function return many values not one, we must use (), ex: ([]string, …etc)

```
func addTask(taskItems []string, newTask string) []string {
    var updatedTaskItems = append(taskItems, newTask)

    return updatedTaskItems
}
```

****

To Create Http Server:

```
package main

import "fmt"
import "net/http"

func main() {
    // in this way we create handle for / without register it in ListenAndServe
    http.HandleFunc("/", helloLoka)
    http.ListenAndServe("localhost:8080", nil);
}

func helloLoka(responseWriter http.ResponseWriter, request *http.Request) {
    var greeting = "Hi, My Name is Jafar Loka. Welcome to Todo project";
    fmt.Fprintln(responseWriter, greeting);
}
**************************************************************

func printTasks(responseWriter http.ResponseWriter, taskItems []string) {

    fmt.Println("List of my Todos")

    for index, task := range taskItems {
        // fmt.Println(index+1, ".", task)
        fmt.Fprintf(responseWriter, "%d. %s\n", index+1, task)
    }
}
**************************************************************
```